Spring Framework

Spring 01 – Intro & Fonctionnalités de base

Bonjour !

Je suis Séga SYLLA

Séga, c’est… le prénom ☺ !

Qui suis-je ?

Qui suis-je ?

- Je suis basé à Nantes
- Je suis CTO / Co-Fondateur d’Ekwi@Tech depuis 2025
- Je suis Développeur / Formateur Java Freelance depuis 2015
- Je pratique JUnit dans un cadre professionnel depuis 2010

Et vous ?

- Quelques mots sur vous ?
- Qu’attendez-vous de cette formation ?

Objectifs du cours Spring (3J)

- Comprendre les fonctionnalités clés de Spring
- Configurer une application Spring
- Découvrir et mettre en œuvre quelques modules

Objectifs du cours de ce support

- Présentation de Spring
- Spring Boot
- Le principe de l’Injection de Dépendances

Spring Framework

A. Introduction au framework Spring

Bref historique de Spring

oct. 2002	mars 2004	nov. 2007	avr. 2014	nov. 2022	oct. 2025
Rod Johnson publie son livre «Expert One-on-One J2EE Design and Development», dans lequel il propose du code, qui va devenir plus tard le framework Spring	Spring 1.0 sort sous licence Apache 2.0	Spring 2.5, avec support des annotations	Spring 4 & Spring Boot	Spring 6 & Spring Boot 3	Spring 6.2.12 & Spring Boot 3.5.7

Qu’est-ce que Spring Framework / Spring Core

Spring est un « framework »

- Boîte à outils Java
- Donne un cadre de travail au développeur pour faire du Java EE (axé entreprise : serveurs, gros projets, etc)
- Spring Framework est souvent abrégé simplement **Spring**
  ➞ désigne l’ensemble des outils fournis par le framework.

Fonctionnalités de base de Spring Framework / Core

Qu’est-ce que permet Spring ?

- Fournir des « **briques** » communes à la plupart des applications Java modernes
- Favoriser les bonnes pratiques (design patterns, architecture, …)
- Spring est **modulaire** : https://spring.io/projects

Avez-vous déjà entendu parler des principes SOLID ?

Principes SOLID

SOLID :

Acronyme qui regroupe 5 principes de conceptions documentés par **Robert C. Martin**.

- **S**ingle responsibility principle : *une entité = un rôle*
- **O**pen/closed principle : *étendre sans modifier l'existant*
- **L**iskov substitution principle : *les sous-classes respectent le contrat de la classe mère*
- **I**nterface segregation principle : *des interfaces fines, pas des interfaces fourre-tout*
- **D**ependency Inversion Principle : *dépendre d'abstractions, pas de détails concrets*

Responsabilité unique *(Single responsibility principle)*

Principe :

- Une unité testable doit avoir une et une seule responsabilité
- Une entité (méthode, classe, module, service) doit être focalisée sur une seule responsabilité métier
- Cela réduit les effets de bord, simplifie les tests, et rend l'évolution plus prévisible

Exemple synthétique :

Une classe ne doit pas à la fois valider une commande, la sauvegarder et envoyer un email.

Ouvert / Fermé *(Open/Closed principle)*

Principe :

- Les entités doivent être ouvertes à l'extension mais fermées à la modification
- On doit pouvoir enrichir un comportement sans modifier le code existant, afin de limiter les régressions
- Cela pousse vers l'utilisation d'abstractions (interfaces, stratégies, décorateurs)

Exemple synthétique :

Plutôt que modifier un calculateur de prix à chaque nouveau type de réduction, on injecte plutôt une **stratégie** de réduction.

Substitution de Liskov *(Liskov substitution principle)*

Principe :

- Les classes dérivées doivent pouvoir remplacer leurs classes mères sans casser le comportement attendu
- Une sous-classe doit respecter le contrat établi par la classe de base
- Les invariants, préconditions et postconditions doivent rester cohérents

Exemple synthétique :

Un *Rectangle* et un *Square* dans une hiérarchie d'héritage : un carré ne peut pas correctement se substituer à un rectangle si les méthodes permettent de modifier indépendamment largeur/hauteur.

Ségrégation des interfaces *(Interface segregation principle)*

Principe :

- Il vaut mieux plusieurs petites interfaces cohérentes (fines) qu'une seule interface générale fourre-tout et lourde
- Les clients ne doivent pas dépendre de méthodes qu'ils n'utilisent pas
- Favorise la modularité et limite les effets de bord

Exemple synthétique :

Au lieu d'une interface *IMachine* avec *print*, *scan*, *fax*, on préfère trois interfaces *Printer*, *Scanner*, *Fax*.

Inversion des dépendances *(Dependency Inversion Principle)*

Principe :

- Les modules de haut niveau ne doivent pas dépendre de modules de bas niveau
- Tous doivent dépendre d'abstractions
- On découple le comportement métier des détails techniques
- Cela facilite la substitution, le test et le versioning

Exemple synthétique :

Un service métier dépend d'un repository via une interface, et non de l'implémentation PostgreSQL.

Modules Spring que l’on va voir

Avec Spring, nous allons construire une application qui va :

- Gérer la persistance de nos données (**Spring Data JPA**)
- Exposer une API REST (**Spring Data Rest**)
- Exposer un front-end simple (**Spring Web MVC** & **Thymeleaf**)
- Gérer l’authentification d’utilisateurs (**Spring Security**)
- Écrire des tests unitaires pour Spring (**Spring Test**)

Fonctionnalités de base de Spring Framework

Dans sa version la plus basique, Spring nous fournit beaucoup d'outils, dont :

- Mise en place du pattern « **Inversion of Control** » (IoC) avec le méchanisme de **Dependency Injection** (DI)
- Pattern « Aspect Oriented Programming » (AOP)
- D'autres comme : Internationalisation (i18n), Validation, gestion des ressources (fichiers locaux)…

Ces fonctionnalités sont rassemblées dans qu'on appelle **Spring Core.**

Spring Framework

B. Spring Boot

Spring vs. Spring Boot

Spring Boot

- Est une « extension » de Spring
- Comme un package Spring pré-configuré pour fonctionner *out of the box*
- **Simplifie la gestion des dépendances** (maven)
- + simple ➞ + rapide & + efficient
- Contient un « serveur » intégré : on a juste à lancer le *main*
- Pour démarrer un projet Spring boot : Spring Initializr

Spring Boot – Gestion des dépendances

Avantage 1 : Spring Boot nous fournit des « starters » qui sont des sortes de *packs* de dépendances :

Spring Boot

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Spring

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-web</artifactId>
    <version>5.3.5</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.5</version>
</dependency>
```

Spring Boot – Gestion des dépendances

Spring Boot

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- Versions déduites de celle choisie pour Spring Boot
- Chaque module a son *starter*

- Liste complète des starters officiels

Spring Boot – Autoconfiguration

Avantage 2

- Spring Boot nous **auto-configure** beaucoup de choses fastidieuses selon les « starters » choisis.
- Il nous crée aussi une classe principale toute prête !

```
@SpringBootApplication
public class SpringDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringDemoApplication.class, args);
    }
}
```

Spring Boot – Autoconfiguration

Annotation qui active **l'autoconfiguration**

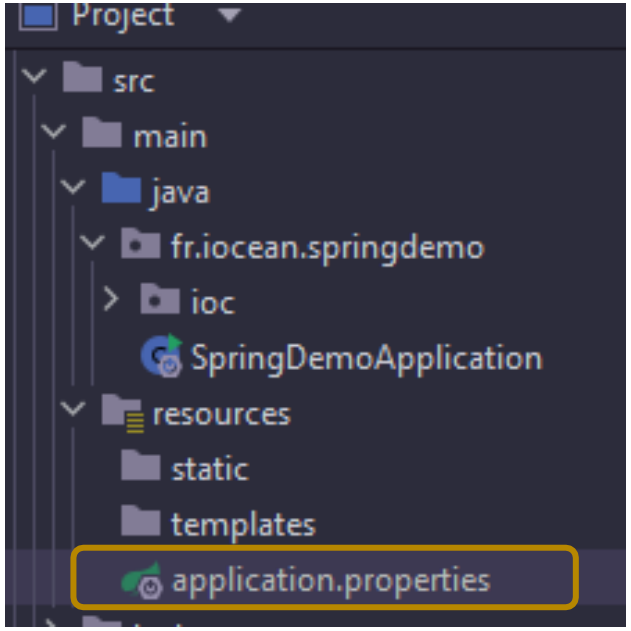
```
@SpringBootApplication
public class SpringDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringDemoApplication.class, args);
    }
}
```

- Fait gagner du temps ➞ le développeur peut se concentrer sur le code "métier" de son application
- La liste des effets de cette auto-configuration est longue.

Spring Boot – Properties

Avantage 3

- Gestion des « propriétés » via le fichier **application.properties**



- Nous permet d'ajouter des paramètres ou des valeurs facilement
- Ces valeurs seront aisément accessibles dans notre code

Spring Boot – Autres avantages

Avantage 4 : Monitoring et Management du programme

- Permet de modifier « à chaud » des éléments sans redémarrage

Avantage 5 : Déploiement via un simple **.jar**

- On a juste besoin d'un JRE installé pour lancer le programme où on veut
- Grâce au **Tomcat** embarqué, pas besoin d'avoir un serveur applicatif

Spring Boot en résumé

Pour résumer

- Est plus simple d’utilisation que Spring seul
- Vient avec une autoconfiguration
- Pas besoin d’un serveur conteneur web tel que Tomcat => déploiement facilité

Spring TP 01

Démarrons un projet Spring Boot !

Objectifs :

- Initialiser un projet Spring Boot
- Faire un Hello World
- Observer la structure de notre projet

Spring Framework

C. Pattern Inversion of Control (IoC)

Pattern « Inversion of Control » (IoC) – Principe

À quoi sert ce pattern Inversion of Control ?

- À réduire les dépendances entre nos composants (classes, …)
- À améliorer la maintenabilité et l'évolutivité de notre application.

Pourquoi nos classes seraient trop dépendantes entre elles ?

(trop "couplées")

Pattern « Inversion of Control » (IoC) – Principe

Identifier les couplages trop forts entre classes :

```
public class Magasin {
    private Produit produit;

    public Magasin() {
        this.produit = new Produit("Quelque chose");
    }
}
```

Le **new** implique de *savoir comment instancier* la classe

- Si on change la façon de créer des Produits : ça nous force à modifier notre usage dans Magasin (**fort couplage** entre les 2 classes)
- Ce n'est pas au Magasin de s'occuper à créer des Produits ! (principe de **responsabilité unique**)

Pattern « Inversion of Control » (IoC) – Principe

Spring utilise la méthode **d'injection de dépendances (DI)** pour mettre en place l'IoC.

- Inversion of Control = **concept**
- Dependency Injection = une **implémentation** de ce concept.

Elle fonctionne sur le « **IoC Container** » de Spring.

- Cette notion d'**IoC/DI** est centrale à Spring
- C'était la principale motivation du framework à ses débuts.

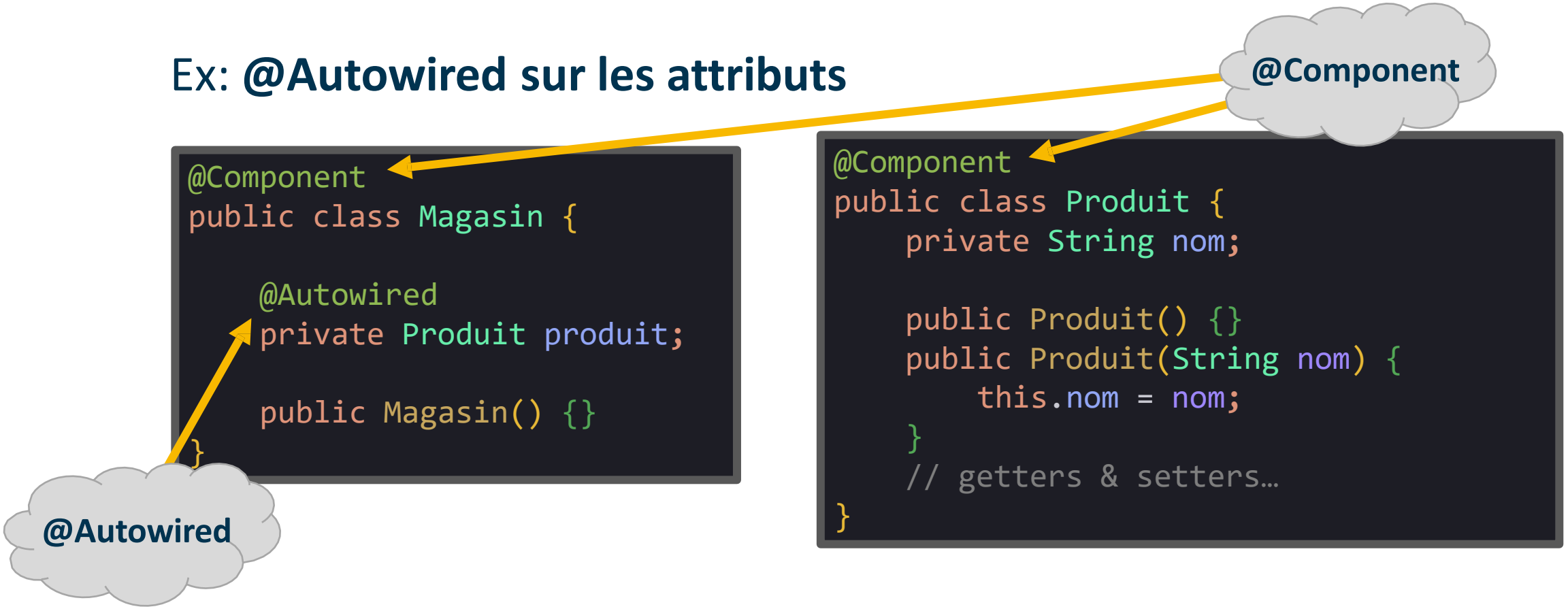
IoC – Utilisation dans Spring

Il y a plusieurs façons d'utiliser la DI avec Spring.

Ex: **@Autowired sur les attributs**

```
@Component
public class Magasin {

    @Autowired
    private Produit produit;

    public Magasin() {}
}
```

```
@Component
public class Produit {
    private String nom;

    public Produit() {}
    public Produit(String nom) {
        this.nom = nom;
    }
    // getters & setters…
}
```



IoC – Utilisation dans Spring

Impact de ces annotations :

```
@Component
public class Magasin {
    { … }
```

@Component

- Se met sur la classe
- Déclare que c'est un **composant Spring (bean)**
- Ce "bean" sera disponible dans ce fameux **IoC Container**

IoC – Utilisation dans Spring

```
{ … }
@Autowired
private Produit produit;
{ … }
```

@Autowired

- Indique à Spring qu'il faut qu'il nous fournisse un Produit déjà construit
- Ce « Produit » doit être un **bean Spring** (il doit donc avoir @Component sur lui)
- On peut désormais utiliser un Produit dans notre classe **sans avoir à l'instancier** ! (il n'y a plus de **new**)

Pattern « Inversion of Control » (IoC) – Résumé

- La **responsabilité** d'aller *chercher et de construire* nos éléments ne repose plus sur la classe qui en a besoin !

- **On a délégué cette responsabilité** à un méchanisme externe spécialisé (ici : Spring et son IoC container)

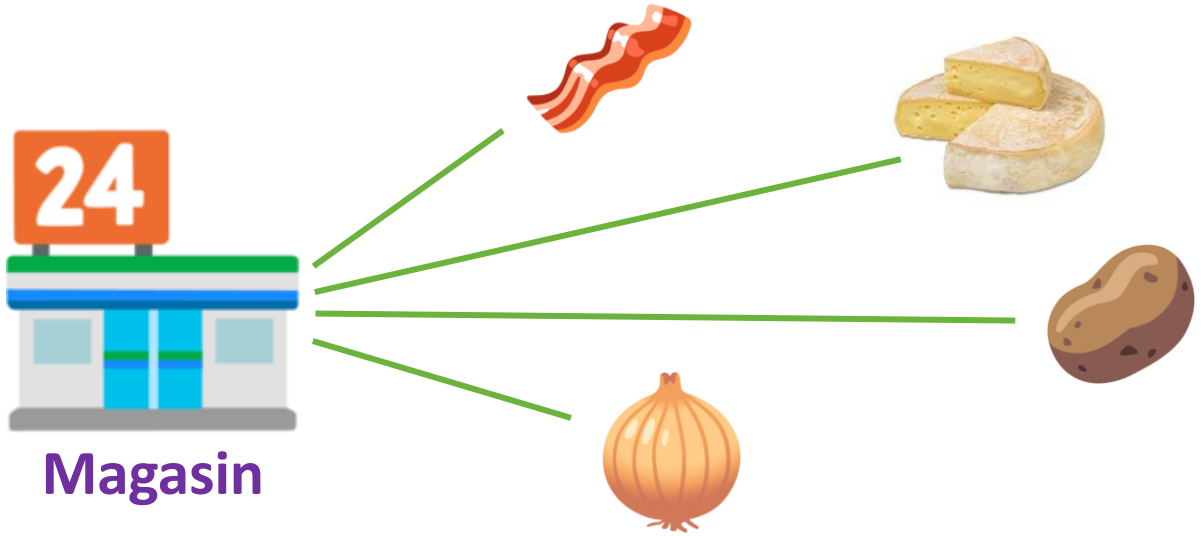
IoC – Analogie du Cuisinier

Un Cuisinier a besoin d'ingrédients pour sa Tartiflette :

- Pommes de terre, Reblochon, lardons, oignons, …

➡ Il ne va pas cultiver ses pommes de terre, faire son reblochon, etc.
➞ Ce n'est **pas sa responsabilité** !

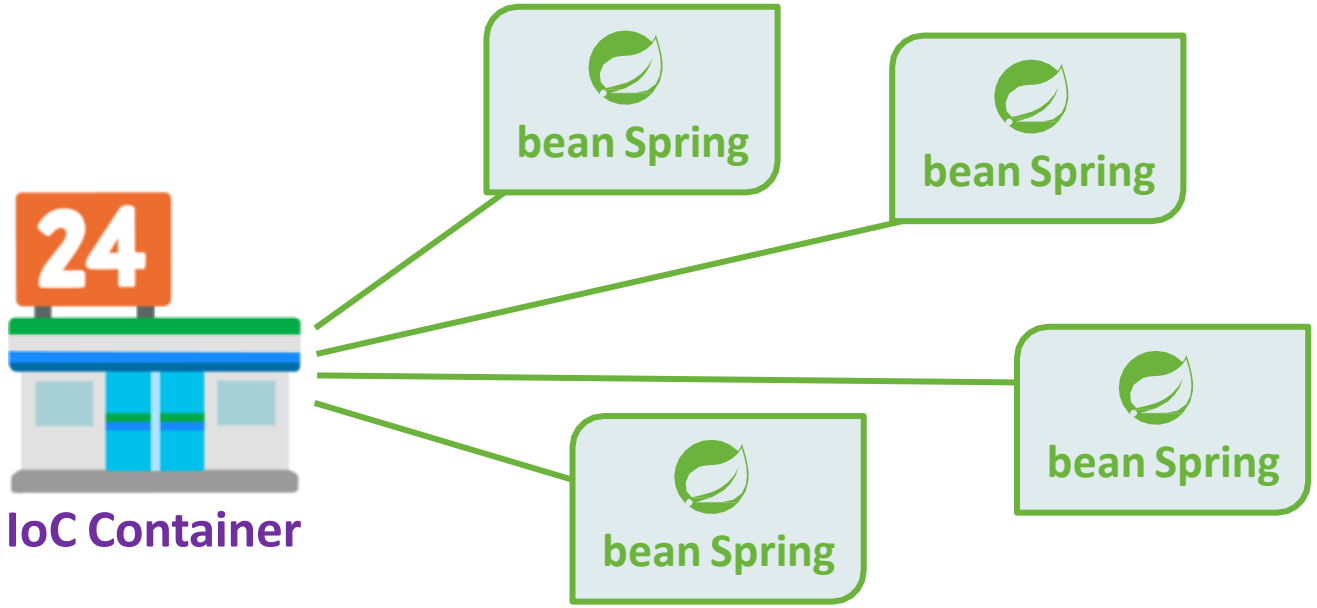
➡ Il va aller **se fournir quelque part** ➞ Plus rapide, pas besoin de savoir faire



IoC – IoC Container et contexte Spring

L'IoC container est aussi appelé **contexte Spring**

- **C'est notre « magasin » de beans !**
- Activé et configuré via l'annotation **@ComponentScan**



IoC – IoC Container et contexte Spring

Avec Spring Boot nous n’avons directement **@ComponentScan**, mais **@SpringBootApplication :**

- C’est un composé de 3 autres annotations.
- **@SpringBootConfiguration**, **@EnableAutoConfiguration et @ComponentScan**

IoC – IoC Container et contexte Spring

```
@SpringBootApplication
public class SpringDemoApplication {
    { … }
```

@SpringBootConfiguration

- La classe sert à configurer notre projet Spring (**classe de configuration**)

@EnableAutoConfiguration

- Active la fonctionnalité d'auto-configuration de Spring Boot

@ComponentScan

- Active le **scanning** des composants Spring **dans le package de la classe et ses sous-packages** (configurable)

IoC – Résumé des annotations

- @SpringBootApplication / @ComponentScan : **déclenche la méchanique d'IoC Container**
- Un élément est **ajouté** dans l'IoC Container via **@Component**
- Un élément est **récupéré** de ce contexte via **@Autowired** (peut être implicite selon la méthode choisie)

IoC – Autres propriétés d'un bean

Autres choses à savoir sur un « bean » Spring :

- **Un bean doit avoir la structure classique getters/setters et constructeur vide !**

Il est soit un :

- **Singleton** (par défaut): un seul *bean* par instance de conteneur
- **Prototype** : une nouvelle instance du *bean* est créée à chaque injection de celui-ci (rarement utilisé)

IoC – Autres propriétés d'un bean

- Un *bean* Spring a **un cycle de vie** qui est géré par le framework (instanciation, injection, destruction…)
- Spring expose des "endpoints" (points d'extensions) dans un bean :

```
{…}
@PostConstruct()
private void postConstruct() {
    // Code exécuté juste après l'instanciation du bean
}

@PreDestroy
private void preDestroy() {
    // Code exécuté juste avant la destruction du bean
}
```

IoC – Autres façons d'utiliser la DI

Propriété / Attribut	Constructeur	Setter

`@Component`
`public class A {`
 	
`  @Autowired`
`  private B b;`
 	
`}`
	`@Component`
	`public class A {`
	
	`  private B b;`
	
	`  @Autowired`
	`  public A(B b) {`
	`  this.b = b;`
	`  }`
	`}`
		`@Component`
		`public class A {`
		
		`  private B b;`
		
		`  @Autowired`
		`  public void setB(B b) {`
		`  this.b = b;`
		`  }`
		`}`

La plus pratique et courante… mais désormais non recommandée !	Permet d'avoir des propriétés immutables	Permet de personnaliser le setter et de gérer des injections optionnelles

Cas de conflit lors d'une injection par type

Scénario :

- Un bean A demande l'injection d'un bean de type B
- 2 beans correspondent au type demandé : B1 et B2
- Spring ne peut pas choisir, l'exécution aboutit à une erreur

Exemple :

```
@Autowired
private PaymentService paymentService;
```

```
@Service
public class PaypalPaymentService implements PaymentService { }

@Service
public class StripePaymentService implements PaymentService { }
```

Résultat :

Une erreur NoUniqueBeanDefinitionException est lancée car le type PaymentService *correspond à* ***plus d'un bean****.*

Comment résoudre le conflit d'injection par type ?

- Rendre un bean prioritaire avec l'annotation ***@Primary***
- Transformer l'injection en une injection par nom avec ***@Qualifier***
- Définir des profils sur les beans *(**@Profile**)* et activer un des profils dans ***application.properties*** via ***spring.profiles.active***

Résoudre le conflit d'injection avec @Primary

L'annotation `@Primary` donne la priorité au bean principal à injecter en cas de conflit détecté

```
@Service
@Primary
public class PaypalPaymentService implements PaymentService { }
```

Résoudre le conflit d'injection avec @Qualifier

L'annotation `@Qualifier` permet d'indiquer explicitement le bean voulu

```
@Autowired
@Qualifier("stripePaymentService")
private PaymentService paymentService;
```

```
@Autowired
private PaymentService stripePaymentService;
```

Résoudre le conflit d'injection avec @Profile

L'annotation `@Profile` permet d'injecter automatiquement un bean selon une variable d'environnement d'exécution

```java
public interface PaymentService {
    void pay(double amount);
}
```

```java
@Service
@Profile("dev")
public class PaypalPaymentService implements PaymentService {

    @Override
    public void pay(double amount) {
        // DEV: simulate Paypal call
        System.out.println("Paypal DEV: processing " + amount);
    }
}
```

```java
@Service
@Profile("prod")
public class StripePaymentService implements PaymentService {

    @Override
    public void pay(double amount) {
        // PROD: real Stripe integration
        System.out.println("Stripe PROD: processing " + amount);
    }
}
```

```
spring.profiles.active=dev
```

Spring TP 02

Démontrer le méchanisme de IoC/DI

Objectifs :

- Créer un *bean* Spring de type « Service »
- L’injecter dans une autre classe et l’utiliser
- Écrire une méthode PostConstruct

Annexes – Liens

- Documentation Spring
- Spring Initializr